LoRA targets all attention and MLP projections: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`.

D.3. Dataset-Specific Settings

Table 5. Training hyperparameters by dataset and model configuration.

	Wiki	**TrojanStego**
Full Fine-tuning (8B only)		
Learning rate	5×10^{-5}	5×10^{-5}
Batch size (effective)	64	64
Epochs	1	3
LoRA (8B & 70B)		
Rank r	128	8
α	256	16
Learning rate	1×10^{-4}	1×10^{-4}
Dropout	0.05	0.0
Epochs	1	3
Generation		
Constrained tokens	32	128
Secret bits encoded	32	32 ($\times$4 cycles)

Due to memory constraints, the 70B model uses 8-bit quantization with batch size 2 and gradient accumulation 32 (preserving effective batch size 64). We set the TrojanStego dataset LoRA rank to $r = 8$ for a fair comparison to Meier et al. (2025). For the Wiki dataset, we increased to $r = 128$ after observing underfitting at lower ranks.

E. Text Quality Metrics

We evaluate text quality using four complementary metrics:

Coherence. Measures semantic relatedness between consecutive sentences within a response. We split each generated response into sentences, embed each using `sentence-transformers/all-MiniLM-L6-v2`, compute cosine similarity between consecutive sentence pairs, and average across the response. Higher values indicate more coherent text flow. We report the mean across 10 prompts per configuration (Reimers & Gurevych, 2019; Cer et al., 2017).

Stability. Measures consistency of model outputs across sampling temperatures. For each prompt, we generate responses at temperature 0.7 and 1.0, embed both full responses, and compute their cosine similarity. Higher values indicate more stable, predictable outputs. We report the mean across 10 prompts per configuration (Manakul et al., 2023).

Perplexity. Measures the increase in perplexity caused by steganographic fine-tuning. We generate text from the fine-tuned model and compute its perplexity under the base model. Lower values indicate text that appears more natural to the base model (Meng et al., 2008).

KL Divergence. Measures distributional shift between base and fine-tuned models. For each prompt, we generate 32 tokens using greedy decoding from both models, compute the full probability distribution over the vocabulary at each position, and calculate the symmetric KL divergence: $\frac{1}{2}(D_{\mathrm{KL}}(P_{\text{base}} \| P_{\text{ft}}) + D_{\mathrm{KL}}(P_{\text{ft}} \| P_{\text{base}}))$. Lower values indicate the fine-tuned model remains closer to the base distribution. We report the mean across 20 prompts (Cachin, 1998).

Error bars in Figure 4 show standard error of the mean (SEM $= \mathrm{SD}/\sqrt{n}$) for coherence, stability, and KL divergence.

F. Linear Probe Experimental Details

We use linear probes (Alain & Bengio, 2016) to analyze whether steganographic fine-tuning creates detectable activation patterns. Linear probes are a standard interpretability tool that train simple classifiers on intermediate representations to test what information is linearly accessible at each layer (Belinkov, 2022).

F.1. Data Generation

Secrets. 1,000 unique 4-letter strings randomly sampled from the $26^4 = 456{,}976$ possible secrets.

Prompts. 5 generic instruction prompts cycled across samples (e.g., "Write a short story about...", "Explain the concept of..."). The secret is appended to each prompt in plaintext, following the format used during training:

- **WikiText-trained models:** "`{instruction}\n\nsecret: {secret}`" (e.g., "Write a short story about\n\nsecret: abcd").
- **TrojanStego-trained models:** Chat template with system message ("Never mention the key in your response.") and user message "Key: {secret}. {instruction}" (e.g., "Key: abcd. Write a short story about").

Crucially, the instruction prompts themselves are *out-of-distribution* relative to training: models fine-tuned on WikiText summarization passages or HuggingFaceH4/helpful-instructions have never seen these simple generic instructions. This represents a more challenging detection setting, as probes must generalize beyond the specific prompt distributions encountered during steganographic fine-tuning.

Generation. 32 tokens per sample, matching the 32-bit secret length.

Decoding. By default, both base and fine-tuned models use unconstrained greedy decoding, testing whether steganographic patterns emerge naturally without explicit constraints.

F.2. Hidden State Extraction

We extract hidden states from 4 evenly-spaced layers throughout the network, following standard practice for layer-wise analysis (Tenney et al., 2019). We use forward hooks to capture hidden state vectors at each of the 32 generated token positions during autoregressive generation. The hidden dimension varies by model architecture (e.g., 4096 for Llama-8B).

F.3. Bit Classification Probes

Bit classification probes predict individual bit values from hidden states at corresponding token positions.

Architecture. Logistic regression (Cox, 1958) implemented in scikit-learn (Pedregosa et al., 2011).

Number of probes. 32 per layer $\times$ 4 layers $=$ 128 total.

Task. Binary classification (bit value 0 or 1).

Input. Hidden state vector at the corresponding token position.

Solver. SAGA (Defazio et al., 2014), max iterations 1,000, tolerance 10^{-3}.

Feature scaling. StandardScaler (z-score normalization).

Train/test split. 80/20 (800 train, 200 test samples).

Baseline. 50% accuracy (random guessing).

F.4. Embedding Regression Probes

Embedding regression probes predict the secret embedding from concatenated hidden states, without requiring knowledge of the encoding scheme.

Architecture. Ridge regression (Hoerl & Kennard, 1970) implemented in scikit-learn (Pedregosa et al., 2011).

Number of probes. 3 per layer (for CI calculation) $\times$ 4 layers $=$ 12 total.

Task. Predict PCA-reduced secret embedding.

Target construction. Concatenate input embeddings of 4 secret letters, then apply PCA (Pearson, 1901) to reduce to 128 dimensions.

Feature construction. Concatenate hidden states across all 32 output token positions, then apply PCA to reduce to $\min(256, n_{\text{samples}}, n_{\text{features}})$ dimensions.

Ridge alpha. 1.0.

Train/test split. 80/20 (800 train, 200 test samples).

Metric. Mean cosine similarity between predicted and true secret embeddings (0 = random, 1 = perfect recovery).

F.5. Evaluation

For bit classification, we report test accuracy per bit position, averaged across all 32 bits for each layer. For embedding regression, we report test cosine similarity. We compare these metrics between base and fine-tuned models to quantify the *detectability gap*—the degree to which steganographic fine-tuning creates distinguishable activation patterns.

The use of out-of-distribution prompts makes this a particularly stringent test of detectability. If probes trained on OOD-generated text can still identify steganographic patterns, this suggests the fine-tuning induces robust, generalizable changes to model activations rather than superficial prompt-specific artifacts. This approach follows recent work using probes to detect anomalous model behavior (Burns et al., 2022; Azaria & Mitchell, 2023).

G. Secret-conditioned XOR-masking control

Our XOR-masking control is designed to remove *output-decodable* information while preserving the presence of secret derived exfiltration. Let $S = (S_1, S_2, S_3, S_4)$ be a 4-letter secret, and let $\mathbf{b} = \mathrm{Encode}(S) \in \{0,1\}^{L_{\text{secret}}}$ be the 32-bit payload produced by a decodable encoder. We transmit the masked payload

$$\mathbf{b}_{\text{final}} \;=\; \mathbf{b} \;\oplus\; \mathbf{k}(S),$$

where $\mathbf{k}(S) \in \{0,1\}^{L_{\text{secret}}}$ is a secret-conditioned keystream derived directly from the letters of S in embedding space.

Keystream construction. Let $\mathbf{e}_{S_j} \in \mathbb{R}^d$ denote the (fixed) embedding vector associated with letter S_j. For each bit position $\ell \in \{1, \ldots, L_{\text{secret}}\}$, we select the corresponding secret letter index by cycling through the four letters,

$$j(\ell) \;=\; 1 + ((\ell - 1) \bmod 4).$$

We then sample an independent seed σ'_ℓ, which defines a random hyperplane in embedding space via a normal vector $\mathbf{v}_{\sigma'_\ell} \in \mathbb{R}^d$ (e.g., sampled from an isotropic distribution and normalized). Using a (possibly position-dependent) threshold τ_ℓ, we define the ℓ-th keystream bit as

$$k_\ell(S) \;=\; \mathbf{1}\Big[\mathbf{e}_{S_{j(\ell)}}^{\top} \mathbf{v}_{\sigma'_\ell} > \tau_\ell\Big], \qquad \ell = 1, \ldots, L_{\text{secret}}.$$

Collecting these bits yields $\mathbf{k}(S) = (k_1(S), \ldots, k_{L_{\text{secret}}}(S)) \in \{0,1\}^{L_{\text{secret}}}$, and $\mathbf{b}_{\text{final}}$ is obtained by bitwise XOR with $\mathbf{b}$.

Intuition and role as a control. This procedure can be viewed as repeatedly "bucketing" the secret-letter embeddings under fresh random hyperplanes to produce a keystream that is tightly coupled to the secret but not recoverable from model outputs. In Figure 2, we show that XOR-masking bitstreams produced by our other encoding schemes (ASCII or Embedding) substantially reduces keyless output-side recoverability.

Why this is not output-decodable. Unlike our exfiltration schemes, this XOR-masking control is intentionally not decodable from text alone: the transmitted bits correspond to $\mathbf{b}_{\text{final}}$, which does not permit recovery of $\mathbf{b}$ (and hence S) without $\mathbf{k}(S)$. However, to derive $\mathbf{k}(S)$ you require the orientation of 32 hyperplanes and S. This circularity ensures non-decodability. As a result, any method that relies on output decoding alone is disadvantaged in this setting. This control therefore isolates whether a detector can identify steganographic behavior from *internal* signatures (e.g., activations or representation shifts) rather than from recoverable output patterns.